Labsheet 3

1. Write a C++ program to implement Bubble Sort in ascending order. Display the array before sorting and after each pass.

```cpp
int main16() {
    int a[100], n;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> a[i];

    cout << "Before sorting: ";
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";
    cout << endl;

    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (a[j] > a[j + 1])
                swap(a[j], a[j + 1]);
        }

        cout << "After pass " << i + 1 << ": ";
        for (int j = 0; j < n; j++)
            cout << a[j] << " ";
        cout << endl;
    }

    return 0;
}
```

2. Write a C++ program to implement Bubble Sort in descending order and display the final sorted array.

```cpp
int main17() {
    int a[100], n;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> a[i];

    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (a[j] < a[j + 1])
                swap(a[j], a[j + 1]);

    cout << "Descending order: ";
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";

    return 0;
}
```

3. Write a C++ program to implement Insertion Sort in ascending order. Display the array after each insertion/pass.

```cpp
int main18() {
    int a[100], n;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> a[i];

    for (int i = 1; i < n; i++) {
        int key = a[i];
        int j = i - 1;

        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }

        a[j + 1] = key;

        cout << "After pass " << i << ": ";
        for (int k = 0; k < n; k++)
            cout << a[k] << " ";
        cout << endl;
    }

    return 0;
}
```

4. Write a C++ program to implement Selection Sort in ascending order. Display the minimum element selected in each pass.

```cpp
int main19() {
    int a[100], n;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> a[i];

    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;

        for (int j = i + 1; j < n; j++)
            if (a[j] < a[minIndex])
                minIndex = j;

        cout << "Minimum in pass " << i + 1 << " = " << a[minIndex] << endl;
        swap(a[i], a[minIndex]);
    }

    return 0;
}
```

5. Write a C++ program to implement Merge Sort using separate functions for Merge Sort and Merge operations.

```cpp
int main20() {
    int a[100], n;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> a[i];

    mergeSort(a, 0, n - 1);

    cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";

    return 0;
}
```

6. Write a C++ program to implement Quick Sort using the last element as the pivot.

```cpp
int main21() {
    int a[100], n;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> a[i];

    quickSort(a, 0, n - 1);

    cout << "Sorted array: ";
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";

    return 0;
}
```

7. Write a C++ program to implement Quick Sort and display the array after each partition.

```cpp
int main22() {
    int a[100], n;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> a[i];

    quickSortDisplay(a, 0, n - 1);

    return 0;
}
```

8. Write a C++ program to implement all five sorting algorithms: Bubble, Insertion, Selection, Merge, and Quick Sort.

```cpp
void printArray(int a[], int n) {
    for (int i = 0; i < n; i++)
        cout << a[i] << " ";
    cout << endl;
}

void bubbleSort(int a[], int n) {
    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++)
            if (a[j] > a[j + 1])
                swap(a[j], a[j + 1]);
}

void insertionSort(int a[], int n) {
    for (int i = 1; i < n; i++) {
        int key = a[i];
        int j = i - 1;

        while (j >= 0 && a[j] > key) {
            a[j + 1] = a[j];
            j--;
        }

        a[j + 1] = key;
    }
}

void selectionSort(int a[], int n) {
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;

        for (int j = i + 1; j < n; j++)
            if (a[j] < a[minIndex])
                minIndex = j;
        swap(a[i], a[minIndex]);
    }
}

int main23() {
    int a[100], b[100], c[100], d[100], e[100], n;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++) {
        cin >> a[i];
        b[i] = c[i] = d[i] = e[i] = a[i];
    }

    bubbleSort(a, n);
    insertionSort(b, n);
    selectionSort(c, n);
    mergeSort(d, 0, n - 1);
    quickSort(e, 0, n - 1);

    cout << "Bubble Sort: ";
    printArray(a, n);
    cout << "Insertion Sort: ";
    printArray(b, n);
    cout << "Selection Sort: ";
    printArray(c, n);
    cout << "Merge Sort: ";
    printArray(d, n);
    cout << "Quick Sort: ";
    printArray(e, n);

    return 0;
}
```

9. Take the same input array and sort it using Bubble Sort, Insertion Sort, and Selection Sort. Compare the number of comparisons and swaps.

```cpp
int main24() {
    int a[100], b[100], c[100], n;
    int bc = 0, bs = 0;
    int ic = 0, is = 0;
    int sc = 0, ss = 0;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++) {
        cin >> a[i];
        b[i] = c[i] = a[i];
    }

    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            bc++;
            if (a[j] > a[j + 1]) {
                swap(a[j], a[j + 1]);
                bs++;
            }
        }
    }

    for (int i = 1; i < n; i++) {
        int key = b[i];
        int j = i - 1;

        while (j >= 0) {
            ic++;
            if (b[j] > key) {
                b[j + 1] = b[j];
                is++;
                j--;
```

```
            }
            else
                break;
        }

        b[j + 1] = key;
    }

    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;

        for (int j = i + 1; j < n; j++) {
            sc++;
            if (c[j] < c[minIndex])
                minIndex = j;
        }

        if (minIndex != i) {
            swap(c[i], c[minIndex]);
            ss++;
        }
    }

    cout << "Bubble: comparisons = " << bc << ", swaps = " << bs << endl;
    cout << "Insertion: comparisons = " << ic << ", shifts = " << is << endl;
    cout << "Selection: comparisons = " << sc << ", swaps = " << ss << endl;

    return 0;
}
```

10. Take the same input array and sort it using Merge Sort and Quick Sort. Compare their time and space complexity.

```
int main25() {
    cout << "Merge Sort: Best O(n log n), Average O(n log n), Worst O(n log n), Space O(n)\n";
    cout << "Quick Sort: Best O(n log n), Average O(n log n), Worst O(n^2), Space O(log n) average\n";
    return 0;
}
```

11. Write a C++ program that allows the user to select any one of the five sorting algorithms using a menu-driven approach.

```
int main26() {
    int a[100], n, choice;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> a[i];

    cout << "1. Bubble\n2. Insertion\n3. Selection\n4. Merge\n5. Quick\n";
    cout << "Enter choice: ";
    cin >> choice;

    if (choice == 1)
        bubbleSort(a, n);
    else if (choice == 2)
        insertionSort(a, n);
    else if (choice == 3)
        selectionSort(a, n);
    else if (choice == 4)
        mergeSort(a, 0, n - 1);
    else if (choice == 5)
        quickSort(a, 0, n - 1);
    else {
        cout << "Invalid choice";
        return 0;
    }

    cout << "Sorted array: ";
    printArray(a, n);

    return 0;
}
```

12. Write a C++ program to sort an array in ascending order and then in descending order using a sorting algorithm of your choice.

```
int main27() {
    int a[100], n;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++)
        cin >> a[i];

    bubbleSort(a, n);

    cout << "Ascending: ";
    printArray(a, n);

    for (int i = 0; i < n / 2; i++)
        swap(a[i], a[n - 1 - i]);

    cout << "Descending: ";
    printArray(a, n);

    return 0;
}
```

13. Write a C++ program to count the number of comparisons performed by Bubble Sort, Insertion Sort, and Selection Sort for a given input.

```
int main28() {
    int a[100], b[100], c[100], n;
    int bc = 0, ic = 0, sc = 0;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter elements: ";
    for (int i = 0; i < n; i++) {
        cin >> a[i];
        b[i] = c[i] = a[i];
    }

    for (int i = 0; i < n - 1; i++)
        for (int j = 0; j < n - i - 1; j++) {
            bc++;
            if (a[j] > a[j + 1])
                swap(a[j], a[j + 1]);
        }

    for (int i = 1; i < n; i++) {
        int key = b[i];
        int j = i - 1;

        while (j >= 0) {
            ic++;
            if (b[j] > key) {
                b[j + 1] = b[j];
                j--;
            }
            else
                break;
        }
        b[j + 1] = key;
```

```
    for (int i = 0; i < n - 1; i++) {
        int minIndex = i;

        for (int j = i + 1; j < n; j++) {
            sc++;
            if (c[j] < c[minIndex])
                minIndex = j;
        }

        swap(c[i], c[minIndex]);
    }

    cout << "Bubble comparisons = " << bc << endl;
    cout << "Insertion comparisons = " << ic << endl;
    cout << "Selection comparisons = " << sc << endl;

    return 0;
}
```

14. Write a C++ program to demonstrate the best-case behavior of Bubble Sort and Insertion Sort using an already sorted array.

```cpp
int main29() {
    int a[100], b[100], n;
    int bubbleComparisons = 0;
    int insertionComparisons = 0;

    cout << "Enter number of elements: ";
    cin >> n;

    cout << "Enter already sorted elements: ";
    for (int i = 0; i < n; i++) {
        cin >> a[i];
        b[i] = a[i];
    }

    for (int i = 0; i < n - 1; i++) {
        bool swapped = false;

        for (int j = 0; j < n - i - 1; j++) {
            bubbleComparisons++;
            if (a[j] > a[j + 1]) {
                swap(a[j], a[j + 1]);
                swapped = true;
            }
        }

        if (!swapped)
            break;
    }

    for (int i = 1; i < n; i++) {
        int key = b[i];
        int j = i - 1;

        if (j >= 0) {
            insertionComparisons++;
        }

        while (j >= 0 && b[j] > key) {
            b[j + 1] = b[j];
            j--;
        }

        b[j + 1] = key;
    }

    cout << "Bubble comparisons = " << bubbleComparisons << endl;
    cout << "Insertion comparisons = " << insertionComparisons << endl;

    return 0;
}
```

15. Write a C++ program to test Quick Sort on sorted, reverse-sorted, and randomly arranged arrays and observe the effect of pivot selection.

```cpp
int main30() {
    int sorted[] = {1, 2, 3, 4, 5};
    int reverse[] = {5, 4, 3, 2, 1};
    int random[] = {3, 1, 5, 2, 4};

    cout << "Sorted array: ";
    quickSort(sorted, 0, 4);
    printArray(sorted, 5);

    cout << "Reverse-sorted array: ";
    quickSort(reverse, 0, 4);
    printArray(reverse, 5);

    cout << "Random array: ";
    quickSort(random, 0, 4);
    printArray(random, 5);

    cout << "Last-element pivot can give O(n^2) on sorted/reverse-sorted input.\n";

    return 0;
}
```